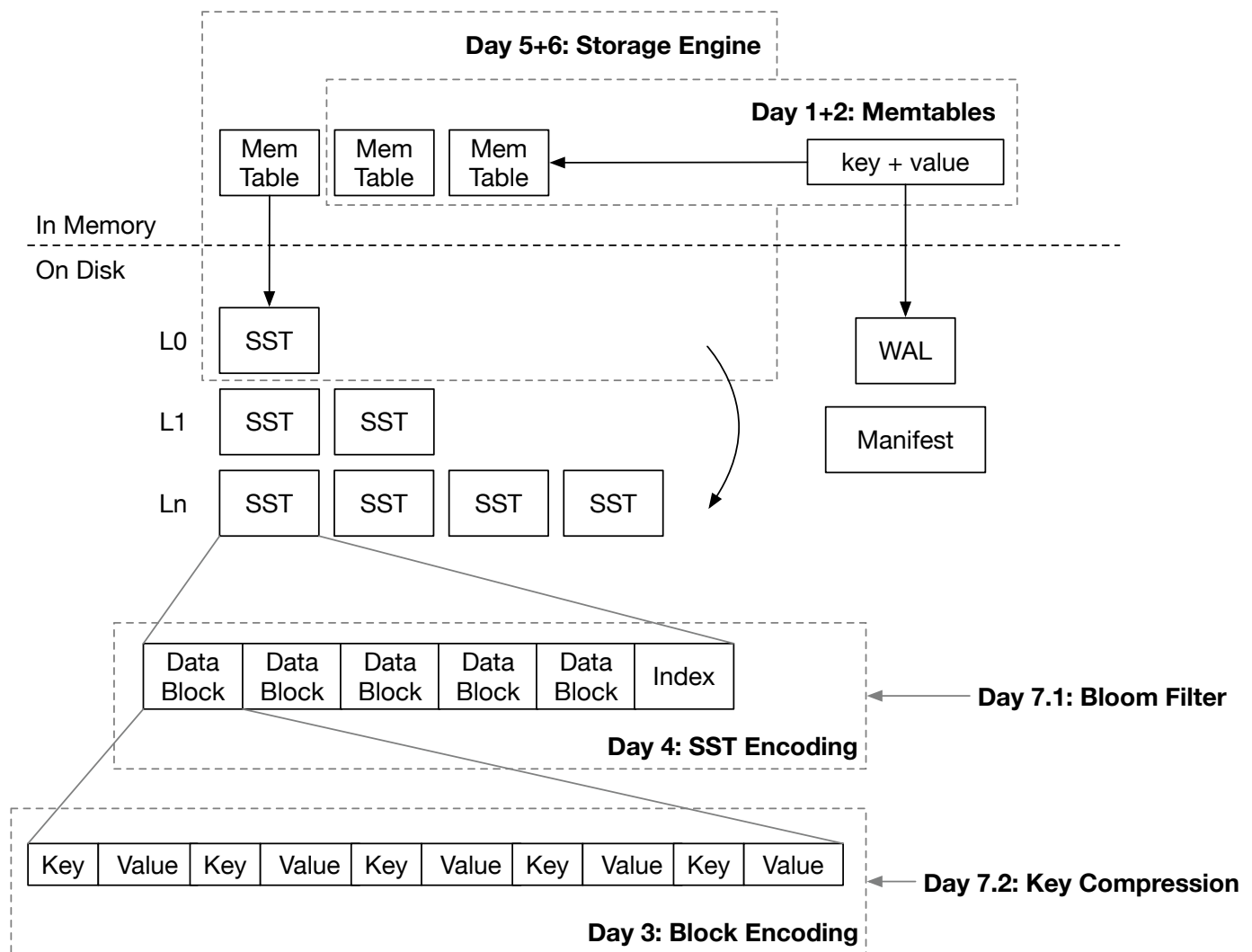


Week 1 Overview: Mini-LSM



In the first week of the course, you will build necessary storage formats for the storage engine, the read path and the write path of the system, and have a working implementation of an LSM-based key-value store. There are 7 chapters (days) for this part.

- **Day 1: Memtable.** You will implement the in-memory read and write path of the system.
- **Day 2: Merge Iterator.** You will extend what you have built in day 1 and implement a `scan` interface for your system.
- **Day 3: Block Encoding.** Now we start the first step of the on-disk structure and build the encoding/decoding of the blocks.
- **Day 4: SST Encoding.** SSTs are composed of blocks and at the end of the day, you will have the basic building blocks of the LSM on-disk structure.
- **Day 5: Read Path.** Now that we have both in-memory and on-disk structures, we can combine them together and have a fully-working read path for the storage engine.
- **Day 6: Write Path.** In day 5, the test harness generates the structures, and in day 6, you will control the SST flushes by yourself. You will implement flush to level-0 SST and the storage engine is complete.

- [Day 7: SST Optimizations](#). We will implement several SST format optimizations and improve the performance of the system.

At the end of the week, your storage engine should be able to handle all get/scan/put requests. The only missing parts are persisting the LSM state to disk and a more efficient way of organizing the SSTs on the disk. You will have a working **Mini-LSM** storage engine.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/mini-lsm.

mini-lsm-book © 2022-2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.